

Advanced programming Assignment

ROLL NO: CSB24065

NAME: Gourisankar Bhuyan

Session: Spring

Year:2026

Question:

Assignment 13: In C, managing strings is a common source of buffer overflows and memory leaks. Implement a Dynamic String Buffer that automatically grows as needed.

Requirements:

1. Create a StringBuffer struct containing a char *data, a size_t length, and a size_t capacity.
2. Write a function sb_init(size_t initial_capacity) that allocates the struct and the data buffer on the heap. Handle NULL returns from malloc.
3. Write sb_append(StringBuffer *sb, const char *str).
4. If the new string exceeds current capacity, use realloc to double the capacity. Ensure you handle realloc safely (don't overwrite the original pointer if it returns NULL).

Write sb_free(StringBuffer *sb) which works as a destructor that frees both the internal data and the struct itself to prevent memory leaks.

Demonstrate the buffer growing at least twice and then free all memory.

Answer:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
```

```
typedef struct {
    char *data;
    size_t length;
    size_t capacity;
} StringBuffer;
```

```
StringBuffer* sb_init(size_t initial_capacity) {

    StringBuffer *sb = (StringBuffer*)malloc(sizeof(StringBuffer));

    if (sb == NULL) {
        printf("Memory allocation failed for StringBuffer\n");
        return NULL;
    }

    sb->data = (char*)malloc(initial_capacity * sizeof(char));

    if (sb->data == NULL) {
        printf("Memory allocation failed for data buffer\n");
        free(sb);
        return NULL;
    }

    sb->length = 0;
    sb->capacity = initial_capacity;

    sb->data[0] = '\0';

    return sb;
}
```

```
void sb_append(StringBuffer *sb, const char *str) {

    size_t str_len = strlen(str);

    while (sb->length + str_len + 1 > sb->capacity) {

        size_t new_capacity = sb->capacity * 2;

        char *temp = (char*)realloc(sb->data, new_capacity);

        if (temp == NULL) {
```

```

        printf("Reallocation failed\n");
        return;
    }

    sb->data = temp;
    sb->capacity = new_capacity;

    printf("Buffer grown to capacity: %zu\n", sb->capacity);
}

strcat(sb->data, str);

sb->length += str_len;
}

void sb_free(StringBuffer *sb) {

    if (sb != NULL) {
        free(sb->data);
        free(sb);
    }
}

int main() {

    StringBuffer *sb = sb_init(8);

    if (sb == NULL) {
        return 1;
    }

    printf("Initial Capacity: %zu\n", sb->capacity);

    sb_append(sb, "Hello");
    printf("String: %s\n", sb->data);
}

```

```

sb_append(sb, " World");
printf("String: %s\n", sb->data);

sb_append(sb, " This is a dynamic string buffer example.");
printf("String: %s\n", sb->data);

printf("Final Length: %zu\n", sb->length);
printf("Final Capacity: %zu\n", sb->capacity);

sb_free(sb);

printf("Memory freed successfully\n");

return 0;
}

```

Screenshots of the output:

The screenshot shows the Visual Studio Code interface with a C file named `ASI1.C` open. The code in the editor includes headers for `stdio.h`, `stdlib.h`, and `string.h`, and defines a `typedef struct` for a string buffer. The terminal window shows the execution output:

```

PS E:\c files> cd "e:\c files\" ; if ($?) { gcc ASI1.C -o ASI1 } ; if ($?) { .\ASI1 }
Initial Capacity: 8
String: Hello
Buffer grown to capacity: 16
String: Hello World
Buffer grown to capacity: 32
Buffer grown to capacity: 64
String: Hello World This is a dynamic string buffer example.
Final Length: 52
Final Capacity: 64
Memory freed successfully
PS E:\c files>

```

The status bar at the bottom indicates the file is at line 87, column 29, with 4 spaces, UTF-8 encoding, and CRLF line endings. The system tray shows the date and time as 14-05-2026, 17:36.